

4.0 Logical Physical Layer

The Logical PHY comprehends the following functions:

- Link initialization, training and power management states
- Byte to Lane mapping for data transmission over Lanes
- Interconnect redundancy remapping (when required)
- Transmitting and receiving sideband messages
- Scrambling and training pattern generation
- Lane reversal
- Width degradation (when applicable)

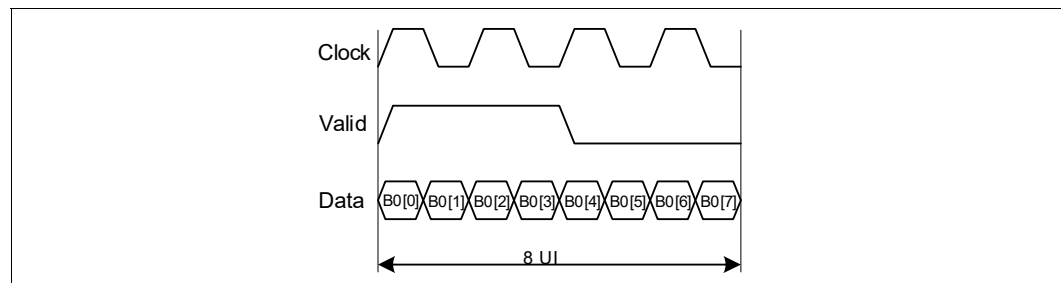
4.1 Data and Sideband Transmission Flow

This specification defines clock, valid and Data to send and receive data over the physical Lanes. The transmitted data is framed by the valid signal.

4.1.1 Byte to Lane Mapping

Data packets are transmitted in Bytes. Within each Byte, bit [0] is transmitted first. [Figure 4-1](#) shows an example of bit arrangement within one byte transmission over Lane 0.

Figure 4-1. Bit arrangement within a byte transfer



Each Byte is transmitted on a separate Lane. Byte 0 (B0) is transmitted on Lane 0, Byte 1 is transmitted on Lane 1 and so on.

[Figure 4-2](#) shows an example of a 256B Flit transmitted over a x64 interface (one x64 Advanced Package module or two x32 Advanced Package modules or four Standard Package modules). If the I/O width changes to x32 or x16 interface (Standard Package), transmission of one Byte per Lane is preserved as shown in [Figure 4-3](#) and [Figure 4-4](#) respectively.

[Figure 4-5](#) shows an example for a width degraded Standard Package module.

Figure 4-2. Byte map for x64 interface

UI \ Lane	0	1	2	3	4	5	6	7	...	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
0 - 7	B00	B01	B02	B03	B04	B05	B06	B07	...	B48	B49	B50	B51	B52	B53	B54	B55	B56	B57	B58	B59	B60	B61	B62	B63
8 - 15	B64	B65	B66	B67	B68	B69	B70	B71	...	B112	B113	B114	B115	B116	B117	B118	B119	B120	B121	B122	B123	B124	B125	B126	B127
16 - 23	B128	B129	B130	B131	B132	B133	B134	B135	...	B176	B177	B178	B179	B180	B181	B182	B183	B184	B185	B186	B187	B188	B189	B190	B191
24 - 31	B192	B193	B194	B195	B196	B197	B198	B199	...	B240	B241	B242	B243	B244	B245	B246	B247	B248	B249	B250	B251	B252	B253	B254	B255

Figure 4-3. Byte map for x32 interface

UI \ Lane	0	1	2	3	4	5	6	7	...	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0 - 7	B0	B1	B2	B3	B4	B5	B6	B7	...	B16	B17	B18	B19	B20	B21	B22	B23	B24	B25	B26	B27	B28	B29	B30	B31
8 - 15	B32	B33	B34	B35	B36	B37	B38	B39	...	B48	B49	B50	B51	B52	B53	B54	B55	B56	B57	B58	B59	B60	B61	B62	B63
16 - 23	B64	B65	B66	B67	B68	B69	B70	B71	...	B80	B81	B82	B83	B84	B85	B86	B87	B88	B89	B90	B91	B92	B93	B94	B95
24 - 31	B96	B97	B98	B99	B100	B101	B102	B103	...	B112	B113	B114	B115	B116	B117	B118	B119	B120	B121	B122	B123	B124	B125	B126	B127
32 - 39	B128	B129	B130	B131	B132	B133	B134	B135	...	B144	B145	B146	B147	B148	B149	B150	B151	B152	B153	B154	B155	B156	B157	B158	B159
40 - 47	B160	B161	B162	B163	B164	B165	B166	B167	...	B176	B177	B178	B179	B180	B181	B182	B183	B184	B185	B186	B187	B188	B189	B190	B191
48 - 55	B192	B193	B194	B195	B196	B197	B198	B199	...	B208	B209	B210	B211	B212	B213	B214	B215	B216	B217	B218	B219	B220	B221	B222	B223
56 - 63	B224	B225	B226	B227	B228	B229	B230	B231	...	B240	B241	B242	B243	B244	B245	B246	B247	B248	B249	B250	B251	B252	B253	B254	B255

Figure 4-4. Byte map for x16 interface

UI \ Lane	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0 - 7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
8 - 15	B16	B17	B18	B19	B20	B21	B22	B23	B24	B25	B26	B27	B28	B29	B30	B31
16 - 23	B32	B33	B34	B35	B36	B37	B38	B39	B40	B41	B42	B43	B44	B45	B46	B47
24 - 31	B48	B49	B50	B51	B52	B53	B54	B55	B56	B57	B58	B59	B60	B61	B62	B63
32 - 39	B64	B65	B66	B67	B68	B69	B70	B71	B72	B73	B74	B75	B76	B77	B78	B79
40 - 47	B80	B81	B82	B83	B84	B85	B86	B87	B88	B89	B90	B91	B92	B93	B94	B95
48 - 55	B96	B97	B98	B99	B100	B101	B102	B103	B104	B105	B106	B107	B108	B109	B110	B111
56 - 63	B112	B113	B114	B115	B116	B117	B118	B119	B120	B121	B122	B123	B124	B125	B126	B127
64 - 71	B128	B129	B130	B131	B132	B133	B134	B135	B136	B137	B138	B139	B140	B141	B142	B143
72 - 79	B144	B145	B146	B147	B148	B149	B150	B151	B152	B153	B154	B155	B156	B157	B158	B159
80 - 87	B160	B161	B162	B163	B164	B165	B166	B167	B168	B169	B170	B171	B172	B173	B174	B175
88 - 95	B176	B177	B178	B179	B180	B181	B182	B183	B184	B185	B186	B187	B188	B189	B190	B191
96 - 103	B192	B193	B194	B195	B196	B197	B198	B199	B200	B201	B202	B203	B204	B205	B206	B207
104 - 111	B208	B209	B210	B211	B212	B213	B214	B215	B216	B217	B218	B219	B220	B221	B222	B223
112 - 119	B224	B225	B226	B227	B228	B229	B230	B231	B232	B233	B234	B235	B236	B237	B238	B239
120 - 127	B240	B241	B242	B243	B244	B245	B246	B247	B248	B249	B250	B251	B252	B253	B254	B255

Figure 4-5. Byte to Lane mapping for Standard package x16 degraded to x8

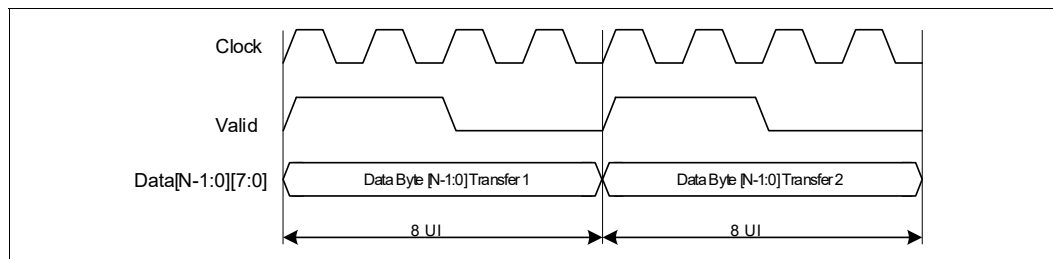
Lane	8	9	10	11	12	13	14	15
Lane	or							
UI	0	1	2	3	4	5	6	7
0 - 7	B0	B1	B2	B3	B4	B5	B6	B7
8 - 15	B8	B9	B10	B11	B12	B13	B14	B15
16 - 23	B16	B17	B18	B19	B20	B21	B22	B23
24 - 31	B24	B25	B26	B27	B28	B29	B30	B31
32 - 39	B32	B33	B34	B35	B36	B37	B38	B39
40 - 47	B40	B41	B42	B43	B44	B45	B46	B47
48 - 55	B48	B49	B50	B51	B52	B53	B54	B55
56 - 63	B56	B57	B58	B59	B60	B61	B62	B63
64 - 71	B64	B65	B66	B67	B68	B69	B70	B71
72 - 79	B72	B73	B74	B75	B76	B77	B78	B79
80 - 87	B80	B81	B82	B83	B84	B85	B86	B87
88 - 95	B88	B89	B90	B91	B92	B93	B94	B95
96 - 103	B96	B97	B98	B99	B100	B101	B102	B103
104 - 111	B104	B105	B106	B107	B108	B109	B110	B111
112 - 119	B112	B113	B114	B115	B116	B117	B118	B119
120 - 127	B120	B121	B122	B123	B124	B125	B126	B127
128-135	B128	B129	B130	B131	B132	B133	B134	B135
136-143	B136	B137	B138	B139	B140	B141	B142	B143
144-151	B144	B145	B146	B147	B148	B149	B150	B151
152-159	B152	B153	B154	B155	B156	B157	B158	B159
160-167	B160	B161	B162	B163	B164	B165	B166	B167
168-175	B168	B169	B170	B171	B172	B173	B174	B175
176-183	B176	B177	B178	B179	B180	B181	B182	B183
184-191	B184	B185	B186	B187	B188	B189	B190	B191
192-199	B192	B193	B194	B195	B196	B197	B198	B199
200-207	B200	B201	B202	B203	B204	B205	B206	B207
208-215	B208	B209	B210	B211	B212	B213	B214	B215
216-223	B216	B217	B218	B219	B220	B221	B222	B223
224-231	B224	B225	B226	B227	B228	B229	B230	B231
232-239	B232	B233	B234	B235	B236	B237	B238	B239
240-247	B240	B241	B242	B243	B244	B245	B246	B247
248-255	B248	B249	B250	B251	B252	B253	B254	B255

4.1.2 Valid Framing

Valid signal is used to frame the transmitted data. For each 8-bit data packet, valid is asserted for the first 4 UI and de-asserted for 4 UI. This will allow data transfer in Raw Format or various Flit Formats as described in [Chapter 3.0](#) using one or multiple valid frames. An example is shown in [Figure 4-6](#) where Transfer 1 and Transfer 2 can be from the same Flit or different Flits.

Note: An 8-UI block assertion is enforced by the Transmitter and tracked by the Receiver during Active state. This means that following the first valid transfer of data over mainband in Active state, each subsequent transfer is after an integer multiple of 8 UI from the rising edge of Valid of the first transfer. Note that for Retimers, this means that the first transfer after entering the Active state cannot be a 'No Flit data transfer + 1 credit release' encoding; this is acceptable because the Retimer-advertised credits are replenished or readvertised whenever the state moves away from Active.

Figure 4-6. Valid framing example



4.1.2.1 Valid Framing for Retimers

The UCIE Retimer releases credits to its local UCIE die using the Valid wire, as described in Table 4-1. Each credit tracks 256 Bytes of data (including any FEC, CRC, etc). The Valid Framing encodings ensure triple bit flip detection guarantee. It is permitted for receiver implementations to trigger Retrain on any bit error (using triple bit flip detection guarantee). It is also permitted for receiver implementations to use the encodings below to correct single bit errors (with the understanding that three bit error detection is lost, and its contribution to overall FIT is negligible).

Table 4-1. Valid framing for Retimers

8-UI Valid ^a	Encoding
00000000b	No Flit data transfer + no credit release
00001111b	Flit data transfer valid + no credit release
11110000b	No Flit data transfer + 1 credit release
11111111b	Flit data transfer valid + 1 Credit release

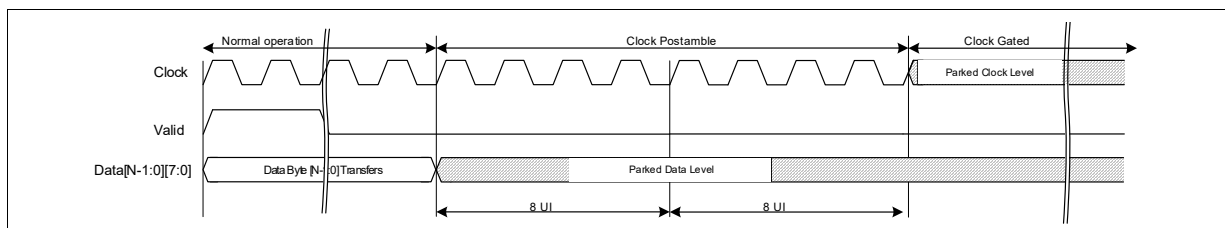
a. Note that the bits above are transmitted on the Link in order from right to left (i.e., bit 0 is transmitted on the Link first, followed by bit 1 and so on until bit 7).

4.1.3 Clock Gating

Forwarded mainband clocks on the UCIE Link must be gated when Valid signal is low after providing fixed 16 UI (8 cycles) of postamble clock for half-rate clocking and 32 UI (8 cycles) of postamble clock for quarter-rate clocking, unless free running clock mode is negotiated or Runtime Recalibration has been requested by the remote Link partner. Data and Clock signal parking levels when clocks are gated are described in Section 5.11.

Note that the clock postamble is required any time that the clock can toggle with Valid assertion, and the clock needs to stop toggling, regardless of LTSM state.

Figure 4-7. Clock gating



4.1.4 Free Running Clock Mode

Free running clock mode is defined as the mode where the forwarded clock remains toggling even when the Transmitter for Valid is held low and there is no data transfer on the interface. This mode must be supported to allow disabling dynamic clock gating for normal operation or debug. This must be negotiated prior to mainband Link training through parameter exchange.

4.1.5 Sideband transmission

Each module supports a sideband interface with a serial data and clock pin pair. Sideband packet formats and encodings are shown in [Chapter 7.0](#) and the electrical characteristics are shown in [Section 5.13](#).

As shown in [Section 7.1.2](#), the sideband message formats are defined as a 64-bit header with no data, with 32 bits or 64 bits of data. A 64-bit serial packet is defined on the I/O interface to the remote die as shown in [Figure 4-8](#). 32-bit data is sent using the 64-bit serial packet with MSBs padded with 0b. Two sideband serial packets on the I/O interface are separated by a minimum of 32 bits low as shown in [Figure 4-9](#). A sideband message with data would be transferred as a 64-bit header followed by 32 bits of low followed by 64-bit data followed by 32 bits of low.

Figure 4-8. Example 64-bit Sideband Serial Packet Transfer

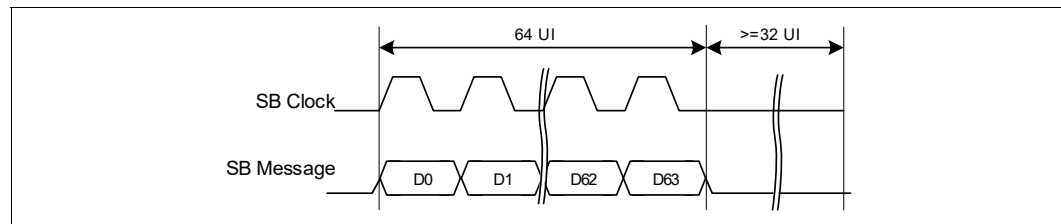
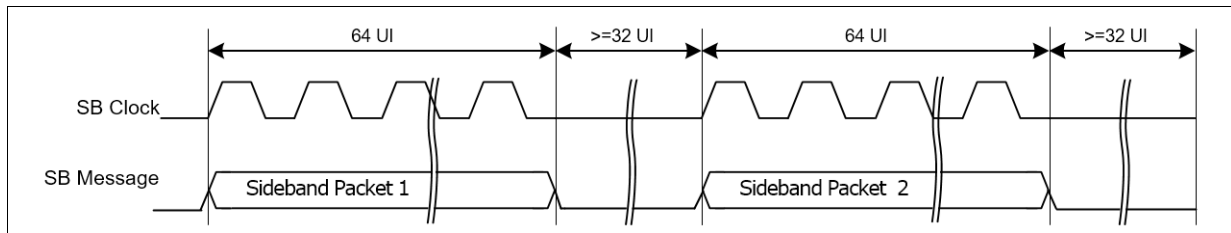


Figure 4-9. Sideband Packet Transmission: Back-to-Back



4.1.5.1 Sideband Performant Mode Operation (PMO)

Sideband designs can negotiate support for 'Sideband Performant Mode Operation (PMO)' by way of the Sideband Feature Extensions (SBFE) mechanism defined in [Section 4.5.3.3.1.1](#). The Sideband PMO bit of the {MBINIT.PARAM SBFE req/resp} sideband message (bit 1) is defined for negotiating this operation (see [Table 7-11](#)). When supporting this feature, a UCIE Link must support this capability on both its transmit and receive direction or not support the capability at all. In a multi-module Link, all modules must advertise the same capability. A UCIE Link must set the Sideband PMO bit to 1 on the {MBINIT.PARAM SBFE resp} sideband message that the UCIE Link transmits, but only if the corresponding Req message also has that bit set to 1 and its receiver is capable of Sideband PMO. Otherwise, the Sideband PMO bit must be cleared to 0.

When Sideband PMO capability is enabled, the 32-UI dead time between the 64-UI data transfers on the sideband is no longer applicable and the sideband can transmit 64-UI data back-to-back with no gaps. See [Figure 4-10](#) and [Figure 4-11](#) for illustration. The transmitter must follow this new mode after the transmitter has sent and received the {MBINIT.PARAM SBFE resp} sideband message with

the Sideband PMO bit set to 1, across all modules. The receiver must be ready to accept packets in this mode after the receiver has transmitted the {MBINIT.PARAM SBFE resp} sideband message with the Sideband PMO bit set to 1. After Sideband PMO is enabled, the transmitter operates in Performant Mode in all states until entry into the RESET state with the SB_MGMT_UP flag cleared to 0. Additionally, PMO can then only be renegotiated on a training sequence with the SB_MGMT_UP flag cleared to 0. Note that from a receiver perspective, due to timing differences, packets might be received without the Sideband Performant Mode even after the chiplet has transmitted an {MBINIT.PARAM SBFE resp} sideband message with the PMO bit set to 1. The sideband receiver must be backward compatible and be able to handle 32 UI of gaps between consecutive 64-UI transfers over the sideband Link.

Figure 4-10. Example 64-bit Sideband Serial Packet Transfer in Sideband Performant Mode

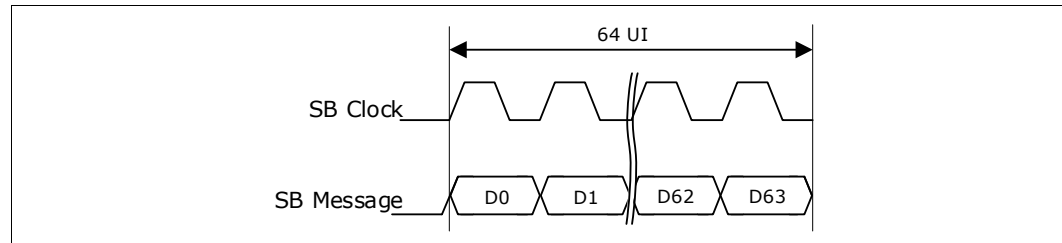
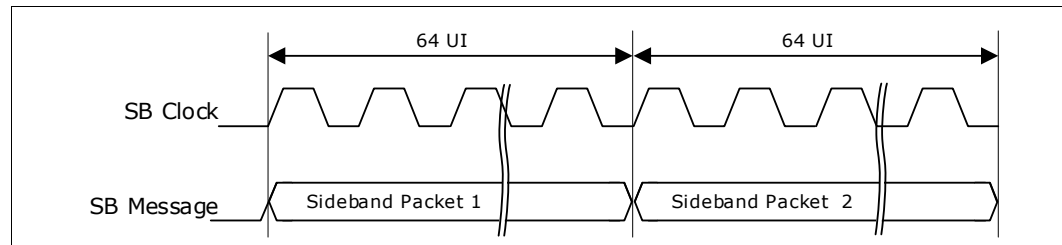


Figure 4-11. Sideband Packet Transmission: Back-to-Back in Sideband Performant Mode



4.2 Lane Reversal

In [Section 4.2](#), [Section 4.3](#), and [Section 4.5](#), the following nomenclature is used:

- **TD_P**: Physical Lane for Data Transmitter
- **RD_P**: Physical Lane for Data Receiver
- **TRD_P**: Physical Lane for Redundant Data Transmitter
- **RRD_P**: Physical Lane for Redundant Data Receiver
- **TD_L**: Logical Lane for Data Transmit
- **RD_L**: Logical Lane for Data Receive
- **TCKP_P**, **TCKN_P** and **TTRK_P**: Physical Lane for Clock and Track Transmitter
- **TCKP_L**, **TCKN_L** and **TTRK_L**: Logical Lane for Clock and Track Transmitter
- **RCKP_P**, **RCKN_P** and **RTRK_P**: Physical Lane for Clock and Track Receiver
- **RCKP_L**, **RCKN_L** and **RTRK_L**: Logical Lane for Clock and Track Receiver
- **TRDCK_P**: Physical Lane for Redundant Clock/Track Transmitter
- **RRDCK_P**: Physical Lane for Redundant Clock/Track Receiver